

SAGESERVE: Optimizing LLM Serving on Cloud Data Centers with Forecast Aware Auto-Scaling

SHASHWAT JAISWAL^{*†}, University of Illinois Urbana-Champaign, USA

KUNAL JAIN^{*‡}, Georgia Institute of Technology, USA

YOGESH SIMMHAN, Indian Institute of Science, India

ANJALY PARAYIL, ANKUR MALLICK, RUJIA WANG, RENEE ST. AMANT, CHETAN BANSAL, VICTOR RUHLE, ANOOP KULKARNI, STEVE KOFSKY, SARAVAN RAJMOHAN, Microsoft, India, UK and USA

Global cloud service providers handle inference workloads for Large Language Models (LLMs) that span latency-sensitive (e.g., chatbots) and insensitive (e.g., report writing) tasks, resulting in diverse and often conflicting Service Level Agreement (SLA) requirements. Managing such mixed workloads is challenging due to the complexity of the inference serving stack, which encompasses multiple models, GPU hardware, and global data centers. Existing solutions often silo such fast and slow tasks onto separate GPU resource pools with different SLAs, but this leads to significant under-utilization of expensive accelerators due to load mismatch. In this article, we characterize the LLM serving workloads at Microsoft Office 365, one of the largest users of LLMs within Microsoft Azure cloud with over 10 million requests per day, and highlight key observations across workloads in different data center regions and across time. This is one of the first such public studies of Internet-scale LLM workloads. We use these insights to propose SAGESERVE, a comprehensive LLM serving framework that dynamically adapts to workload demands using multi-timescale control knobs. It combines short-term request routing to data centers with long-term scaling of GPU VMs and model placement with higher lead times, and co-optimizes the routing and resource allocation problem using a traffic forecast model and an Integer Linear Programming (ILP) solution. We evaluate SAGESERVE through real runs and realistic simulations on 10 million production requests across three regions and four open-source models. We achieve up to 25% savings in GPU-hours compared to the current baseline deployment and reduce GPU-hour wastage due to inefficient auto-scaling by 80%, resulting in a potential monthly cost savings of up to \$2.5 million, while maintaining tail latency and meeting SLAs. The workload traces, our simulator harness and the SAGESERVE scheduler are available at <https://github.com/shashwatj07/SageServe>.

CCS Concepts: • **General and reference** → **Evaluation; Performance; Experimentation; Design**; • **Computing methodologies** → **Cooperation and coordination; Simulation tools; Systems theory; Discrete-event simulation; Distributed simulation; Simulation evaluation**.

Additional Key Words and Phrases: LLM Inference Serving, Scheduling, Forecast Aware Auto-Scaling, Resource Allocation, Workload Analysis, LLM Inference Simulator, Production Traces

^{*}Equal contribution

[†]Work done as an intern at Microsoft

[‡]Work done as a Research Fellow at Microsoft

Authors' Contact Information: Shashwat Jaiswal, sj74@illinois.edu, University of Illinois Urbana-Champaign, USA; Kunal Jain, kjain324@gatech.edu, Georgia Institute of Technology, USA; Yogesh Simmhan, simmhan@iisc.ac.in, Indian Institute of Science, Bangalore, India; Anjaly Parayil, Ankur Mallick, Rujia Wang, Renee St. Amant, Chetan Bansal, Victor Ruhle, Anoop Kulkarni, Steve Kofsky, Saravan Rajmohan, aparayil@microsoft.com, Microsoft, India, UK and USA.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 2476-1249/2025/12-ART61

<https://doi.org/10.1145/3771576>

ACM Reference Format:

Shashwat Jaiswal, Kunal Jain, Yogesh Simmhan, and Anjaly Parayil, Ankur Mallick, Rujia Wang, Renee St. Amant, Chetan Bansal, Victor Ruhle, Anoop Kulkarni, Steve Kofsky, Saravan Rajmohan. 2025. SAGESERVE: Optimizing LLM Serving on Cloud Data Centers with Forecast Aware Auto-Scaling. *Proc. ACM Meas. Anal. Comput. Syst.* 9, 3, Article 61 (December 2025), 24 pages. <https://doi.org/10.1145/3771576>

1 Introduction

Motivation. Recent years have seen rapid adoption of *Large Language Models (LLMs)* in enterprise products and services to power both proactive and user-initiated intelligent features [8]. As their capabilities expand, LLM usage is growing exponentially across enterprise, consumer, and scientific applications [11, 24, 56]. The growth of Agentic AI and workflows is only accelerating this, with LLM agents enabled with tool execution autonomously completing complex tasks [18, 31, 55].

Cloud-hosted, GPU-accelerated Virtual Machines (VMs) are central to scaling LLM inference, prompting major investments from Cloud Service Providers (CSPs) for both internal use and public offerings. AWS UltraClusters offer 100,000 of Trainium2 accelerators used by Anthropic and end-users [51] while Microsoft Azure Cloud’s Eagle system with H100 GPUs features at #5 in the Top500 supercomputing list [46] and Google has introduced the top-end NVIDIA HGX B200 GPUs into its data centers [9]. However, maximizing return on these expensive resources is critical. Misalignment between GPU provisioning and traffic distribution across regions can lead to Service Level Agreement (SLA) violations or resource wastage at either extremes. The need to meet SLAs and provide a smooth user experience often causes CSPs to over-provision GPU capacity, raising infrastructure costs, increasing prices for users, and diverting resources from R&D [26, 37]. This happens at multiple levels: from routing user requests across data centers, to scaling the VM instances and models, and routing within a region to meet the demand [13, 19, 44], to optimizing the execution for a single model instance across GPUs [14, 35, 52]. We focus on the former problems.

Challenges. Unlike VM and container auto-scaling for traditional cloud workloads [17, 40, 48], scaling GPU VMs for LLM workloads presents unique challenges. Commercial platforms like Google Gemini [3], Microsoft Copilot [2], and OpenAI ChatGPT [1] serve a mix of models and inference workload tiers that can broadly be categorized as: (a) *Interactive workload (IW)* that are latency-sensitive and require real-time responses, within seconds, e.g., chatbots, LLM powered search, content moderation; and (b) *Non-interactive Workload (NIW)* requests, which are less time-critical and focus on serving resource-intensive or batch processes, e.g., report writing, data annotation, and simulations within 10s of minutes or hours. These workloads and their priority vary by time, region, and user type (enterprise/consumer), making it difficult to design a unified auto-scaling policy that efficiently handles diverse models and SLA requirements.

Dynamically scaling LLM model instances just-in-time can be *ineffective*, due to traffic variations, and *slow*, blocking GPUs for many seconds or minutes during cold starts, when loading large LLMs [6], e.g., Meta’s Llama2-70B model [41] is $\approx 140\text{GB}$ in FP16, and OpenAI’s GPT models are estimated to be even larger. Reactive scaling that instantiates new VMs and model instances based on real-time metrics, such as incoming *Tokens Processed per Second (TPS)*, can cause over- or under-provisioning if they fail to account for TPS variance and LLM loading delays. E.g., Fig. 1 illustrates a scenario where the model instance has a capacity to serve 4000 TPS. In the first plot, the reactive strategy decides to scale up the instance count at $T = 15$ mins, which causes the instance become available only at $T = 20$ mins due to cold start, resulting in SLA violations for 5 mins due to *under-allocation*. If we had use a conservative instance capacity of 3500 TPS, the reactive approach is susceptible to *over-allocation*, as seen in the second plot, where we unnecessarily scale up at $T = 25$ mins due to a small increase in traffic even though the input TPS later stabilizes.

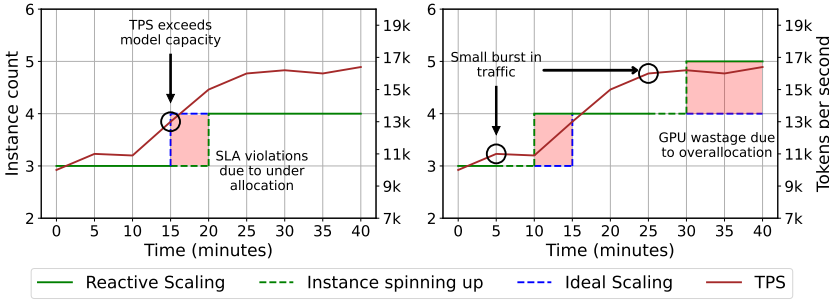


Fig. 1. VM instance scaling (left Y axis) based on incoming TPS (right Y axis) using ideal (pink) and reactive (green) strategies. Shaded region shows the difference in their instance counts.

There is a pressing need for a flexible, lightweight auto-scaling policy that adapts to dynamic workloads, minimizes model loading delays, and reduces costs while meeting diverse SLAs. Besides a knowledge of the LLM serving infrastructure and pipeline design, this also requires access to workload traces and an analysis of their characteristics.

Gaps. Meta [37] discuss the presence of daily peaks, off-peaks, and unpredictable spikes in LLM inference workloads, and validates the presence of both IW and NIW. They emphasize that achieving cost-effective solutions at scale requires extensive benchmarking and production-level insights. However, a lack of public production traces causes literature to often rely on synthetic or regionally scoped datasets that lack key attributes [4, 35, 50]. In this work, we extensively characterize cloud-scale LLM workloads with different SLAs, and will also place the traces in the public domain. While there are routing and scaling strategies proposed for general workloads and LLMs, they make simplifying assumptions. Jain et al. [19] assume identical LLM types and workloads of equal priority, focusing on load balancing across multiple instances within a region. TAPAS [44] is also orthogonal, focusing on the power and thermal characteristics of LLM workloads in a region. Others [13, 36] leverage additional storage and memory capacity for faster model loading and live request migration. Chiron [36] also explores mixing interactive and non-interactive requests and proposes various instance scaling solutions. However, these focus on regional-level deployments, overlooking inter-region imbalances and the load disparities across LLM types within a region. We address these macro challenges across regions as well, to holistically to utilize available capacity. Model optimization strategies [14, 35, 52] also complement our work at the model instance level.

Approach. We first perform a *principled characterization* of real-world LLM inferencing workloads at Microsoft’s Office 365 group (O365), which is one of the largest users of LLMs within Microsoft Azure through its Copilot capabilities, serving over $\approx 10M$ LLM requests per day across the major US data centers. We use diverse metrics to highlight the spatial and temporal features, and predictable request patterns in these workloads for key LLM models. We use both recent (Jul, 2025) and past (Nov, 2024) traces from 3 US data centers, charting the evolution of these workloads. We also identify differing SLAs based on latency and priority for the top inferencing applications, and segregate them into Interactive Fast (IW-F) and Normal (IW-N), and Non-Interactive (NIW) workloads.

We then describe the *current inference serving design*, consisting of request routing to regions, routing to a model instance endpoint within that region, and then their local execution on a model deployment (Fig. 2). We highlight the limitations of the current *siload approach* (Fig. 7a), where separate GPU pools are maintained for IW and NIW requests, which leads to significant under-utilization of the IW resources during off-peak hours. This motivates the design of our proposed scheduler, SAGESERVE, that introduces a *reactive heuristic* over a unified pool of GPU resources that are shared by all workload types (Fig. 7b). It intelligently queues and releases NIW requests to under-loaded GPUs to save GPU-hours while meeting SLAs. To address mismatches

between the *inferencing request load* and the available capacity of different LLM *model instances* (e.g., GPT-4.1, Llama4-Scout), SAGESERVE then applies a *predictive heuristic* (Fig. 7c), formulating a constrained optimization problem for scaling LLM instances based on ARIMA-based times-series forecasts of inference requests [43]. Subsequently, a *reactive scaling heuristic* based on GPU memory utilization – a proxy for load – is used to address minor traffic fluctuations. These improve GPU utilization, ensure SLA compliance to maintain tail latency, and allow surplus capacity from O365 to be leveraged for additional LLM services, e.g., Azure OpenAI Service. SAGESERVE is validated using a realistic simulation harness that we have developed by extending SplitWise [35], using 4 popular open-source LLMs, for week-long traces from multiple US data centers across two time periods, and also compared against Chiron [36], a State of the Art (SOTA) baseline.

Our characterization study is unique in examining large-scale LLM inferencing workloads in an operational system, highlighting opportunities for optimizing such serving pipelines. SAGESERVE is designed for real-world deployment, intelligently leveraging predictive scaling and scheduling strategies to meet the SLAs, and delivering practical benefits.

Contributions. We make the following specific contributions in this paper:

- (1) We describe the current LLM serving platform at Microsoft O365, which routes requests and manages GPU VMs in siloed resource pools across regions to handle diverse workloads (§ 2). We then offer a principled characterization of these workload tiers using diverse metrics for the top IW and NIW applications in US regions (§ 3).
- (2) We highlight the resource inefficiencies of siloed resources for workload tiers, which motivates the need for a systematic approach to GPU VM and LLM instance scaling with continuous optimization (§ 4). We propose SAGESERVE, a unified framework to serve diverse LLM inference workloads across cloud regions, aiming to meet SLAs while maximizing GPU resource efficiency. We formally specify this as an optimization problem (§ 5), and leverage traffic forecasting models and consider practical factors such as instance capacity (in TPS), IW demand, NIW headroom, provisioning overheads, and surplus donations to public services in our design (§ 6).
- (3) We implement a prototype of SAGESERVE and evaluate it using a realistic simulation harness based on SplitWise [35]. We compare SAGESERVE with a SOTA auto-scaler, *Chiron* [36], for two week-long real-world traces (10M requests, 3 regions, 4 LLMs). We are able to reduce GPU VM usage by 25% through improved utilization, and VM cold-start by 80% without violating SLAs. These translate into potential savings of US\$2.5M per month.
- (4) Lastly, we place the workload trace data, scheduling logic and realistic simulator for validating them as open-source artifacts at <https://github.com/shashwatj07/SageServe>. These can serve as a testbed for the wider community.

We also discuss related work (§ 8) and offer our lessons learned and conclusions (§ 9).

2 System and Application Model

We describe the cloud system, LLM deployment and workloads, motivated by global installations at Microsoft O365.

2.1 Cloud VMs and LLM Model Instances

The CSP's system model comprises multiple data centers (*regions*) connected with high bandwidth network, with $\approx 50ms$ inter-region latency. Regions are assumed to be in USA (e.g., US-West, US-Central, etc.) to avoid issues of data sovereignty. Each region has thousands of *GPU VMs*, e.g., Azure ND, with exclusive access to GPUs like NVIDIA A100/H100 or AMD MI300X and all its server resources. These can host LLM instances within regional capacity limits.

There are several standard LLM *model types* that are available, e.g., Llama 2/3.3/4, GPT 3.5/4, Bloom, etc. with associated default weights or custom weights based on fine-tuning (Fig. 2). A

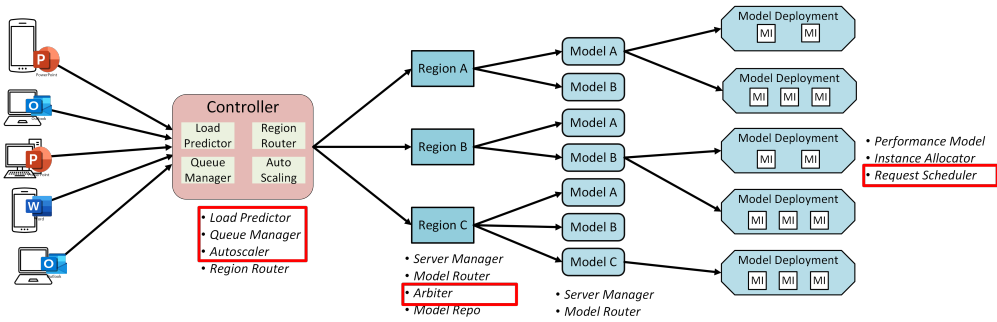


Fig. 2. Overview of the Microsoft O365 LLM serving architecture. The components added or improved upon (Load Predictor, Queue Manager, Autoscaler/Arbiter and Request Scheduler) by SAGESERVE are outlined.

model instance is one copy of a *model type* that can serve requests. Each instance may require multiple GPUs depending on the size of the LLM, e.g., GPT3 may need 9 H100s while Llama-3 needs 4 H100s [25]. Each VM is exclusively to one LLM instance. There can be multiple instances of an LLM type in a region as part of a deployment. A *model endpoint* for that region receives requests to a model type and routes it to one of the instance deployment in a round robin manner. There can be constraints on the minimum and maximum instance counts per endpoint, for robustness and to avoid one model dominating.

Tokens per Second (TPS), the sum of input and output tokens processed per second forms the key throughput capacity metric, and we focus on input TPS that a model instance can serve. The VM type and model type will determine the instance’s performance, defined as input TPS achieved at a target latency [25]. E.g. Llama2-70B [41] and Bloom-176B [7] models can achieve Q1–Q3 performance of 68–293 TPS and 50–177 TPS respectively on 8× Nvidia A100 GPUs; this improves to 95–522 TPS and 82–397 TPS respectively with 8× H100 GPUs.

2.2 LLM Workload Tiers and SLAs

Microsoft O365 supports multiple LLM inference workloads. *Interactive Workloads (IW)* with *low latency constraints* ($O(\text{seconds})$) from client-facing applications like chatbots, code generation, and email suggestions, require “fast” serving. Within these, some may require an even faster serving (IW-F) ($< 1\text{s}$) while others may have a normal interactive latency goal (IW-N). In contrast, *Non-Interactive Workloads (NIW)*, such as nightly document summarization on enterprise repositories or deep content generation, have *relaxed deadlines* ($O(\text{hours})$), and tolerate “slow” (or “no”) serving.

For IW tier, clients for each product or service, e.g., Copilot for Word, Teams etc., may use one or more pre-defined LLM types with thousands of input/output tokens per request, with tens of thousands of daily clients. For simplicity, we assume all clients are US-based since the regions are in the US. NIW also uses a set of pre-defined LLMs whose architectures often overlap with IW but have a lower and non-periodic request rate that is stable through the week. We discuss these in § 5.

IW and NIW tiers have *different* SLAs defined. *Time to First Token (TTFT)* is the time between receiving a prompt to emitting the first response token and indicates responsiveness. In contrast, *End-to-End (E2E) time* is the time taken for generating all output tokens for a request, and *influences* both latency and throughput. IW primarily have TTFT as SLA, typically $< 1s$ for IW-F and $< 1min$ for IW-N at the 95th percentile (P_{95}), but can also have an E2E SLA. The SLA for NIW is typically a *deadline for batch completion* (e.g. 24h to summarize a document repository) with less emphasis on per-request latency. We assume that serving an IW request within its latency SLA accrues a *utility* for the CSP, and serving an NIW request before its deadline has a (*lower*) *utility*.

2.3 Request Routing, Scaling and Scheduling Layers

Routing Mechanisms. All IW requests are routed through a common LLM API service [30] to one of several LLM endpoints that can serve them (Fig. 2). This *global routing* to one of the available regions is based on network latency, spatial proximity or the current load on the region's endpoints. Then, a *region router* sends requests to endpoints for that model within that region, and further to instances within the selected deployment in a round robin manner to balance the load and address token skews. We assume a managed network and trusted security environment. There are no other security constraints that limit the mapping of instances to VM or requests to endpoints.

Scaling Delays. Creating a new LLM instance on VMs when scaling up a model type in a region (Fig. 7) has several *provisioning costs* that can vary based on the conditions. Allocating VMs to the instance is an initial cost. If these VMs do not have the LLM already deployed on them, the model architecture and weights need to be copied and instantiated on them. This cold-start time depends on the model size, and on whether they are available in a local repository in that region (e.g., $\approx 10\text{mins}$), or need to be moved from a remote region (e.g., $\approx 2h$). If the VMs already have the LLM architecture deployed from a prior provisioning but with different weights, only the weights need to be updated and the latency reduces. When an instance is being provisioned, the VMs are not available for use. So the model provisioning time constitutes *wasted GPU cycles*. There are also latency costs to acquire a GPU VM and update all upstream services such as load balancer, etc. Since this total time takes mins–hours, frequent re-provisioning is inefficient.

Donating to Spot Instances. The workloads are executed on LLM instances that are provisioned in a private network space for O365. However, if the endpoints of common Azure LLM model types are idle, they can be leased to external users of Azure AI as (preemptible) *spot LLM instances* for inferencing at a lower cost, and reclaimed when the internal demand increases. Switching an instance from a private to a spot role, and the reverse, is relatively fast, $\approx 1\text{min}$. Typically, the *utility benefits* of leasing out spot instances is lower than that gained from executing the internal IW and NIW workloads. But this is still better than keeping the VMs idle. During some periods, 25% of instances in a region may be donated to spot LLM instances; this is a lost opportunity cost we aim to fix by re-routing and running NIW workloads on them.

Scheduling Algorithms. Each LLM instance employs a scheduling policy that selects the next batch of requests from the waiting queue. The scheduler has access to deterministic request properties including prompt token count, service level agreement (SLA) tier, and arrival timestamp, utilizing these attributes alongside available GPU memory to make batching decisions. To minimize computational overhead from redundant processing, requests remain non-preemptible within a batch until memory exhaustion occurs on the virtual machine.

3 Workload Characterization

In this section, we analyze traces for IW-F, IW-N and NIW workload tiers for O365 applications collected for 1 week each in July, 2025 and November, 2024. Specifically, we examine request traces from four popular OpenAI models deployed in three regions: US East, US West and US Central. To maintain anonymity, we refer to the models as Model A, B, C and D. The traces are made publicly available. Model A has a relatively larger number of parameters compared to Models B, C and D. We focus on the following key dimensions: (1) workload demand from different tiers, characterized in terms of Requests Per Second (RPS) and Tokens Per second (TPS); (2) latency trends; and (3) capacity utilization trends. TPS includes sum of input and output tokens per second.

Demand Patterns. Fig. 3a and Fig. 3c illustrate the cumulative load (RPS and TPS) from the 4 models and 3 regions, for each of the 3 tiers. The Nov 2024 trace does not distinguish between IW-F and IW-N as that is a recently introduced feature, reflecting the evolving nature of LLM-based

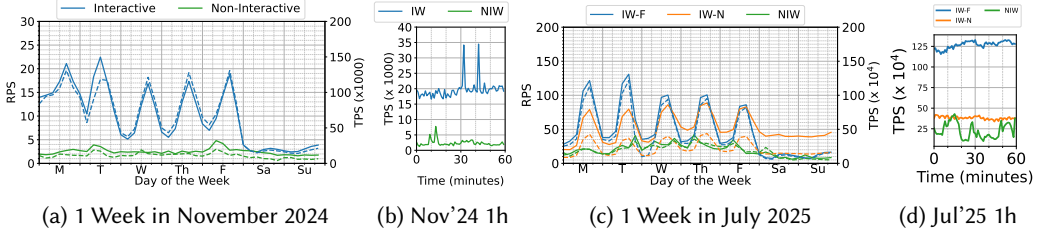


Fig. 3. Aggregated RPS (solid line) and Total Input+Output TPS (dashed line) for IW & NIW for 3 US regions.

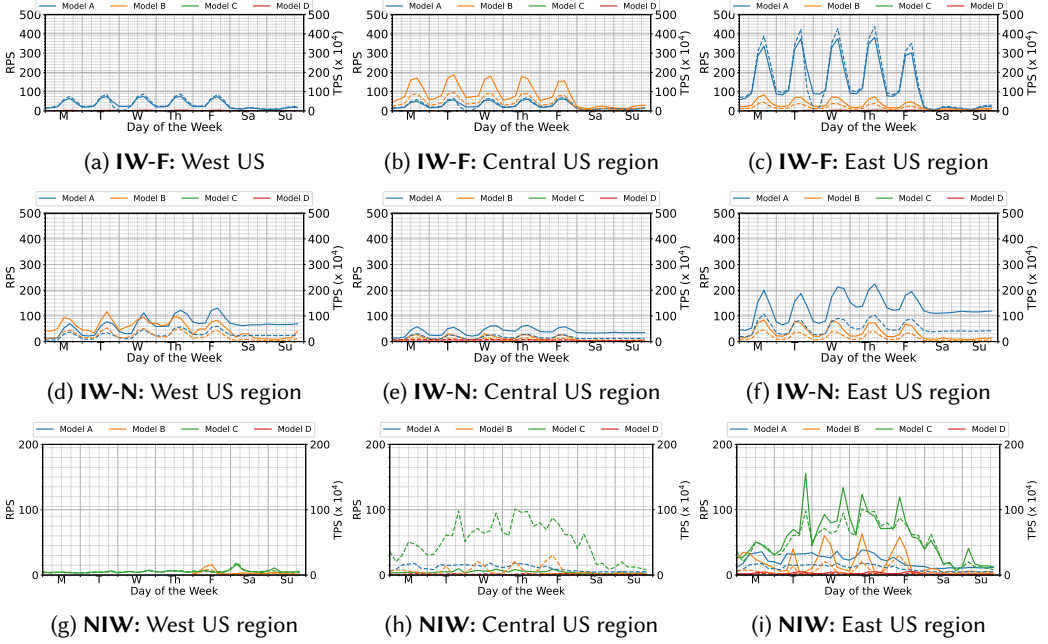


Fig. 4. RPS (solid line) and Total Input+Output TPS (dashed line) per model for 1 week in July 2025, for IW-F, IW-N, and NIW requests (rows) in three US regions (columns).

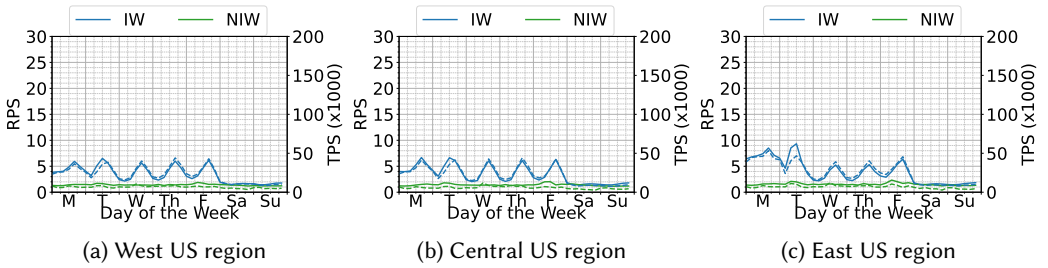


Fig. 5. RPS (solid line) and Total Input+Output TPS (dashed line) of IW and NIW requests, summed across 4 LLM models for 1 week in Nov, 2024 for three regions (a)–(c).

offerings. *IW-F* workloads exhibit clear diurnal periodicity with weekends quiescing, indicating a high degree of predictability. The periodicity score (ps) of Model A varies from 0.7–0.95, indicating

a strong positive autocorrelation between daily traffic patterns. *IW-N traces* also show periodicity, with trends similar to *IW-F*, having $ps \approx 0.6\text{--}0.8$ for Model A and $0.3\text{--}0.5$ for Model B. In contrast, *NIW traces* are less predictable ($ps = 0\text{--}0.286$), without clear periodicity. Notably, *IW-F* form the largest fraction of all requests and TPS, followed by *IW-N*, which together are 72% of all requests.

Next, we analyze the properties of requests from the different models in each tier and region, as shown in Fig. 4a–Fig. 4i for the *Jul, 2025 trace*. As observed from the cumulative load in Fig. 3c, *IW-F requests* exhibit this on a per-region basis too and for all models. However, the amplitude of the traffic patterns vary by region, with East US exhibiting a high demand, followed by Central US and West US. Since requests from any US client can be routed to any of the US data centers by the Region Router (Fig. 2), based on the load and latency, this does not necessarily reflect the client demand from those regions. For the same reason, we do not see any timezone phase shifts in load across the three regions. Inter-region routing is enabled by low network latency between regions and makes the problem more challenging due to the need for global optimization. *IW-N requests* also exhibit periodicity across most regions (Fig. 4d–Fig. 4f). However, in some cases, it shows growth in usage across different weekdays, e.g., Model B has more requests on Wed/Thu/Fri than Mon/Tue, visible in East and West US. In general, model demand for *IW* workloads varies by region, while maintaining the overall trend. E.g., Model A (blue lines) is most popular in East US, at 4× the load as West US for *IW-F*, whereas Model B (orange) sees the highest demand in Central US (for *IW-F*) and West US (*IW-N*). This could be due to the imbalance in the capacity allocated to each LLM type per region. Consequently, the capacity required for each model differs across regions.

In contrast, *NIW requests* are less predictable on TPS and RPS, and the request demand is lower in all regions, with West US having negligible requests. The popularity of *NIW* models also vary across regions. Interestingly, the TPS per request for Model C in Central US is much higher compared to other workloads due to the presence of a feature evaluation and testing application. This is due to the presence of batch-oriented workloads in this tier, which perform bulk operations.

Summary. Latency-sensitive *IW* requests exhibit strong diurnal and weekday-versus-weekend periodicity within a region for each model. This pattern should be leveraged to design systems that optimize resource allocation for anticipated demand. The lower load, poor periodicity but flexible SLA for *NIW* makes them suitable for opportunistic, deferred execution.

Evolution of Workloads Over Time. Fig. 3a and Fig. 3c compare the shift in cumulative workload trends across the two traces 7-months apart, and similarly Fig. 4 and Fig. 5 for each region. Both the request rate and TPS have increased during this time by $\approx 5\times$. New workloads include conversational agents, RAG scenarios, and feature testing and evaluation frameworks, driven by the increasing adoption of Copilot in O365 applications and the emergence of new LLM workloads and applications. This happens across regions as well, while still retaining the periodicity. This underscores the critical need for improved capital efficiency of GPU resources in the LLM serving design, all the more so given the long lead times for acquiring new hardware and the need to conserve operational costs while delivering SLA. In addition, the workload tiers have also evolved during this period with the introduction of fast and normal latency tiers within the interactive workload. This can further evolve into a continuum from fast to slow, and high to low priority workloads over time.

Summary. As new workloads to emerge with varying latency needs and priorities, the serving system must scale efficiently – without leading to a disproportionate increase in capacity needs.

Application-level Workload Patterns. Fig. 6a shows the generic names of the top 10 applications in O365 leveraging LLMs, based on the request count from the day of the week with the highest traffic, cumulatively across all tiers. Notably, 41.2% of the requests originate from a Retrieval-augmented Generation (RAG) system, which explains the high number of tokens processed per second. Other top applications include context-driven use cases such as insights generation, content creation, chat

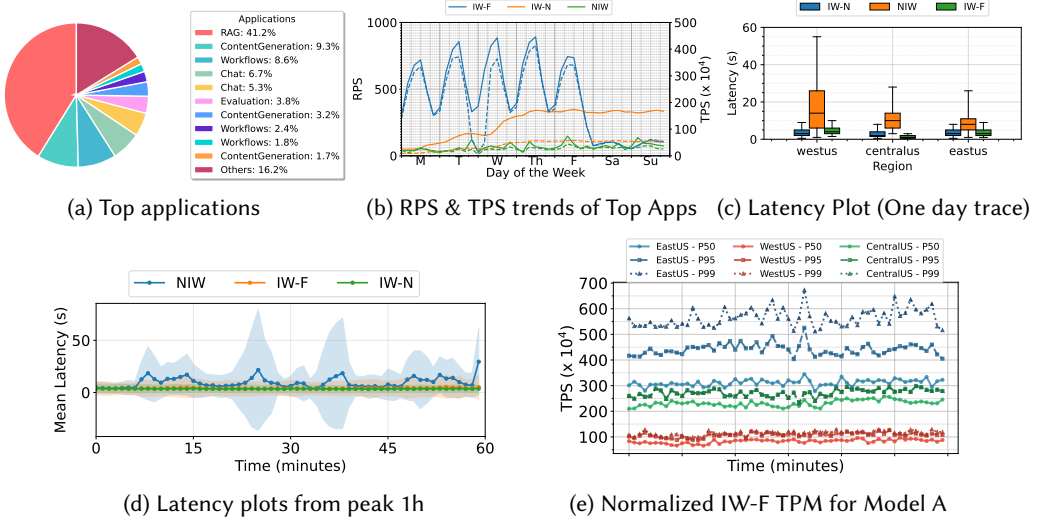


Fig. 6. (a) Top applications across different workloads. (b) RPS and TPS of top applications over 24h. (c) E2E request latency for workloads and regions. (d) Latency variation for the peak 1h of the cumulative. (e) Trans. per Min. (TPM) at at different percentiles from among model deployments within each region for 1h, indicating diversity of load. All for July 2025 trace.

applications, and evaluation frameworks. Fig. 6b reports the RPS and TPS trends for just these top application across different latency tiers. As seen earlier, latency-sensitive IW-F requests continue to have a diurnal pattern while IW-N shows a gradual growth, both of which are predictable using simple forecasting models we later discuss. Additionally, NIW requests maintain a consistent load throughout the week. The mean TPS for the NIW workloads is $\approx 177.5 \times 10^4$ TPS, indicating a relatively stable load throughout the observed time period.

While the daily trends across 10s of minutes or hours exhibit patterns, this does not carry forth at *fine-grained time scales*. Fig. 6c shows the latency distribution across various tiers for the three regions, aggregated across the top applications. NIW workloads exhibit greater variance in latency, as expected, since they have a relaxed SLA with mean latency of 15s and variance of $120s^2$. The variance ratio of NIW:IWF:IW-N is 150 : 1 : 0.5. The E2E latency trends vary by region, with IW-F showing region-specific variability that indicates load imbalance. E.g., West US has (mean, median, P95 latency) of (4455ms, 2700ms, 14,539ms), while Central US shows (3349ms, 618ms, 14,967ms), and East US (3615ms, 1840ms, 11,179ms). We zoom in and analyze the load (Fig. 3d, Fig. 3b) and latency (Fig. 6d) during the peak hour of the day. These plots reveal noticeable latency spikes at short time scales of 1min, highlighting greater variability in IW-F latency compared to IW-N. E.g., the blue area plot in Fig. 6d shows the standard deviation in the latency for IW-F, which changes a lot indicating potential load imbalance. This imbalance is also seen from the load served by model deployments within a region (Fig. 6e), where P50, P95 and P99 load are reported across time per region for Model A. While West US (orange) and Central US (green) serve similar loads using different instances, East US (blue) exhibits divergence across the P50–P99 latencies, indicating uneven load distribution across model instances in that region. This imbalance is further corroborated by the fact that East US accounts for 52% of the cumulative capacity allocated to Model A in the three regions, followed by West US (27%) and Central US (21%). These highlight the need for fine-grained resource scheduling within regions.

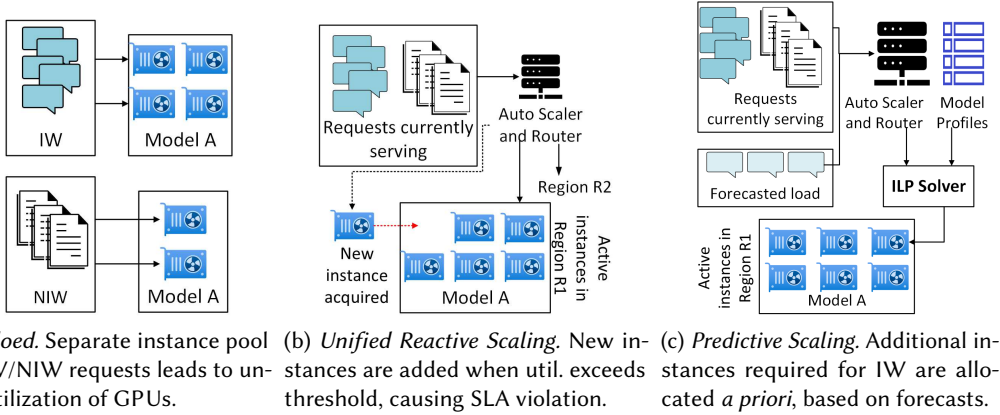


Fig. 7. Routing and Scaling Strategies for LLM Inference

4 Approach to Effective Resource Scaling

We study the effect of scaling resources allocated to an LLM type on GPU capital efficiency and SLA compliance for both IW and NIW workloads. Since GPU VMs are costly, the goal is to maximize their utility – either by serving IW or NIW tiers, or leasing the idle capacity as spot instances. The baseline *siloed deployment* is currently used in Microsoft O365 for serving LLM inference requests (Fig. 7a). It maintains separate instance pools per workload (IW/NIW) and LLM type in each region. It applies a greedy scaling policy, adding instances to the region endpoint when the effective memory utilization for the instances increases $> 70\%$, and returns an instance to spot pool if the utilization drops $< 30\%$. The effective memory utilization excludes memory used for model weights, and is a reliable proxy for the request load as it essentially captures the KV Cache footprint of in-flight requests. These scaling decisions are made per request, constrained by instance limits per model type. However, siloing into pools fragments VMs, often leading to suboptimal utilization.

As an alternative, we propose a *reactive scaling heuristic* to serve both IW and NIW tiers from a *unified resource pool* for a model in a region (Fig. 7b), allowing dynamic sharing of spare capacity across workload types [36]. Further, NIW requests are queued and served only when instance utilization drops below a limit (e.g., 50–60%) or when the SLA deadline for the request is approaching. This ensures that they leverage spare capacity without affecting IW performance or triggering a instance scale-up. This enables shared use of model instances across workload tiers and improves overall VM utilization, rather than donate to spot instances. Switching an LLM instance between spot and internal endpoints takes 1 min, while changing the model hosted on a VM requires ≈ 10 mins. We trigger scaling based on the effective utilization seen at regional endpoints, with a 15-second cooldown enforced between events.

We illustrate the *siloed* and *unified reactive* scaling heuristics for 4 open-source LLMs: Bloom, Llama 2, Llama 3.1, Llama 3.2, deployed in the 3 US regions. Each region has 20 instances per model: 16 for IW and 4 for NIW in the siloed approach, and all 20 in a unified pool with the unified heuristic. Here, we do not separate out IW-F and IW-N. The minimum and maximum instance counts per endpoint are 2 and 3, which is the current practice in O365, ensuring fault tolerance and load balancing. We use a realistic simulation harness built on Splitwise [35], whose results closely match real-world behavior (see § 7.1). We use $8 \times$ A100s per instance, and replay 1 day of workload requests from West US (Tue of Nov, 2024 in Fig. 5), which has 1.4M IW and 0.2M NIW requests.

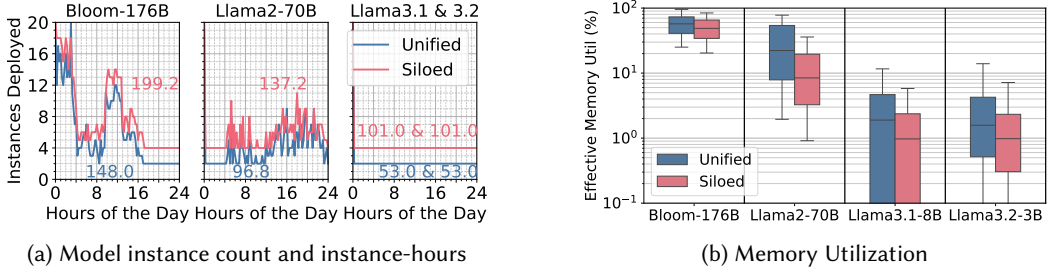


Fig. 8. Performance of Unified vs. Siloed strategies for workload trace in West US from Tuesday of Nov, 2024 (Fig. 5a), with peak 20 instances per model. Unified uses 34.5% fewer instance-hours than the Siloed.

Table 1. 95%ile of TTFT and E2E latencies for serving different models using siloed and unified approach.

Metric	Bloom-176B		Llama2-70B		Llama3.1-8B		Llama3.2-3B	
	Siloed	Unified	Siloed	Unified	Siloed	Unified	Siloed	Unified
TTFT (s)	14.5	12.9	34.9	34.5	1.0	1.0	1.0	1.0
E2E (s)	55.3	53.3	98.3	99.1	10.6	10.5	19.2	18.9

Fig. 8a shows the instance count at West US every 15-mins for the Siloed and Unified approaches for each model for that day. Our Unified deployment strategy consumes fewer model instance hours (area under the curve, shown as text label) for all four models despite using similar scaling thresholds for both deployments. This is because the instances in Unified are not tied to a workload type (IW or NIW) and can be used dynamically by either, based on demand. Since Llama 3.1 and 3.2, being lighter in size, process tokens much faster, they maintained the minimum instance count throughout. Siloed allocates 2 instances each for IW and NIW while Unified shares the same 2 instances among IW and NIW requests.

This consolidation is reflected in the higher memory utilization for the Unified heuristic (Fig. 8b), while not sacrificing the SLA latency for IW, e.g., with the change in P_{95} TTFT staying within 12% for Bloom and almost identical for Llama (Table 1). Thus, both process the full trace within the SLA, but Unified uses fewer resources and also donates 52 instance-hours to spot, compared to Siloed. The memory utilization of Llama 2 is generally less than Bloom, indicating over allocation for Llama models. The Unified pool can take better advantage of this by re-dedicating GPUs from Llama to Bloom (inter-model scaling) and adapt better to the complementary demands compared to the Siloed approach, which does not allow allocation of VMs across models. Hence, using a unified pool of instances for IW and NIW can improve resource utilization and cost efficiency, opening new optimization avenues for flexible NIW processing. However, reactive scaling used by itself can affect the SLA of IW due to under-allocation, or raise the costs due to over-allocation (Fig. 1). It is also sensitive to fine-grained temporal variations (Fig. 3b). This motivates the need for predictive scaling that leverages TPS predictions for the workloads coupled with the Unified resource pool (Fig. 7c), complemented by enhanced load balancing within a region. Next, we formulate this as an optimization problem (§ 5) and then describe the architecture and heuristics of SAGESERVE (§ 6).

5 Optimization Problem

We define an optimization problem for inference request routing and capacity allocation, to serve fast and slow workloads within the required SLA while maximizing utilization.

Definition. Given a captive set of VMs of specific types in multiple regions,

Table 2. Variables used in optimization problem

Symbol	Type	Description	Symbol	Type	Description
l	int	Model types	$\theta_{i \times k}$	[float] $_{l \times g}$	TPS provided by model i on GPU k
r	int	Number of regions	α_k	[float] $_g$	Cost of acquiring VM with GPU k
g	int	GPU types	$\sigma_{i \times k}$	[float] $_{l \times g}$	Cost of starting an instance of model i on GPU k
$n_{i,j,k}$	[int] $_{l \times r \times g}$	Instances of model i at region j running on GPU k	$\delta_{i,j,k}$	[int] $_{l \times r \times g}$	ILP output: optimal number of changes in VM allocation
$\rho_{i,j}(w)$	[int] $_{l \times r}$	TPS requested for model i from clients at region j for future time window w			

- we need to continuously ensure that the right number of model instances of different model types are provisioned as endpoints at the regions, and
- route the incoming workload across these endpoints,

such that

- we maximize the utility of the workload requests completed within their SLA, and
- maximize the capacity utilization of the VMs for the interactive workloads.

We have two parts to solving this. *First*, we need to **optimally provision instances** for model endpoints in different regions to handle this workload, within the available VM capacities. We also need to consider the overhead for (re)provisioning an LLM instance set onto VMs. So, VM reprovisioning is only viable at a coarse granularity, e.g., every 15 mins, reclaiming spot instances can be fast, e.g., each 1 min. *Second*, we need to **route requests to the model instances** in different regions while meeting the SLA. These routing decisions can use real-time information on the load and responsiveness of the region endpoints. We use tools like the SplitWise simulator [35] to estimate the latency for serving requests at a certain request rate to ensure we meet the SLA, and ARIMA for load predictions [43]. Spare model instances can be released to spot. Next, we define these as an optimization problem. Table 2 has the notations.

Constraints. Say, the current number of VMs with GPU k assigned to a model i at region j at a given time is $n_{i,j,k}$. Let $\delta_{i,j,k}$ be the optimal number of changes to be made to this VM count to service all IW requests in the next hour. When *servicing IW within a region*, say the forecasted TPS from clients for model i at region j during each time window w over the next decision making window of, say, 1 hour, is $\rho_{i,j}(w)$. Say each model type in a region must serve at least $0 < \epsilon \leq 1$ fraction of its peak future request load in real-time. Excess load $(1 - \epsilon)$ can be rerouted to other regions to reduce the number of model-instance changes needed. This is given by: $\sum_k (n_{i,j,k} + \delta_{i,j,k}) \times \theta_{i,k} \geq \max_w \epsilon \times \rho_{i,j}(w) \forall i, j$. When *servicing IW across all regions*, we must ensure that all requests for a model i received from all regions j can cumulatively be processed using its model instances across all region, with rerouting, i.e., $\sum_j \sum_k (n_{i,j,k} + \delta_{i,j,k}) \times \theta_{i,k} \geq \max_w \sum_j \rho_{i,j}(w) \forall i$. Also, we should never deallocate more models than are allocated, $\delta_{i,j,k} \geq -n_{i,j,k}$.

Objective. Subject to the above constraints, we minimize the wasted resource overheads when provisioning VMs and instances required for IW workloads while meeting their SLAs. We incur two overheads when provisioning a new model instance: (i) VM start up cost to instantiate a new VM (γ), and (ii) the deployment cost (μ) when loading the model and its weights on a VM, where $\gamma = \sum_k (\alpha_k \times \sum_{i,j} \delta_{i,j,k})$ and $\mu = \sum_k \sum_i \sum_j (\sigma_{i,k} \times \max(0, \delta_{i,j,k}))$. These new VM are unusable during this period, making spot instances more attractive. Given these, our objective is: **arg min($\gamma + \mu$)**. Our formulation accounts for multiple models and regions, while allowing region re-routing or using GPUs with different throughputs for cost optimization. We assign values to $\theta_{i,k}$ by benchmarking model i on GPU k , α_k using publicly available GPU VM costs and $\sigma_{i,k}$ as the product of α_k with the average start up time of the model i on GPU k .

Finding Optimal Solutions. Since all the decision variables such as # of changes in VM count are integers, we can solve this using ILP. While the runtime of an ILP solver can grow exponentially with the variables, we expect these parameters to be tractable. E.g., the average solver runtime in our experiments with $l = 4$, $r = 3$ and $g = 1$ was 1.41s. Increasing this with l and r to 20 and g to 5 raises the solver time to 33s. These are acceptable for hourly decisions.

6 Architecture of SAGESERVE

Our SAGESERVE LLM serving architecture (Fig. 2) provides APIs similar to other LLM serving platforms, including real-time streaming API for IW that returns outputs once each token is generated [28, 34] while using a Batch APIs [29, 33] for NIW requests, which takes requests and returns responses asynchronously. The global router receives a request and routes it to the relevant region hosting the LLM instances which can service the request (§ 6.1). Others orchestrate the forecasting and optimization logic (§ 6.3), scaling logic (§ 6.4 and NIW queue manager (§ 6.2).

6.1 Routing Logic

The routing module of SAGESERVE (Fig. 2) decides routing of requests to regions by the *Region Router*, routing to endpoints within a region, and routing locally among the instances of an endpoint.

Global Routing for IW Requests. When IW requests are received at the Region Router, the region routing logic checks the effective memory utilization of all the available regions hosting the model and routes the request to the region with memory utilization less than a given threshold, e.g., 70%, based on production systems at Microsoft O365. If multiple regions match, we can specify an order of preference, e.g., based on network proximity. The *effective memory utilization* is calculated as the ratio of the *sum of the effective memory utilized* to the *effective memory available* across all instances for a model in a region. The effective available memory for a model instance is obtained by subtracting model weights from the total VM memory. If none of the preferred regions has memory utilization less than the threshold, then the region with the least memory utilization among the choices is selected.

Global Routing for NIW Requests. NIW requests are sent by the Region Router to a *Queue manager* that holds these requests (Fig. 2). Each model endpoint in a region periodically send a signal to the Queue Manager when their effective utilization falls below a specific threshold (60%, in our experiments). Then, the NIW requests waiting in the queue for that region and model are incrementally removed by the Queue Manager and routed to the available endpoint (§ 6.2).

Routing Logic at Region Endpoint. For both IW and NIW requests, the *Model Router* routes requests arriving at a region to the least loaded deployment endpoint for that model, based on the effective memory utilized. Requests are sent to the instance with the minimum remaining tokens to process, based on the “Join The Shortest Queue Logic” [16].

Scheduler at the Model Instance. A local queue is maintained at the instance as well, and its scheduler batches inference requests in a first-come, first-served manner. IW requests arriving at an instance are assigned priority 0 and available immediately for inference. NIW requests may also arrive with a priority 0, as set by the Queue Manager when their deadline is approaching, and are considered on par with IW requests. Otherwise, NIW requests have a default priority 1, and are selected for inference only if there are no priority 0 requests ahead in the queue.

6.2 Queue Manager for NIW

The *Queue Manager* (Fig. 2) asynchronously routes NIW requests to specific regions and endpoints. Upon receiving a capacity availability signal from a model endpoint, async feedback logic pulls queued requests for that model type and routes them to the signaling region. NIW requests have a default 24-hour deadline. Requests with age < 10 hours receive priority 1, while those > 10 hours

receive priority 0, similar to IW requests. If the signal indicates regional endpoint effective memory is less than 60%, one request is sent; if less than 50%, two requests are sent. These thresholds are tunable hyper-parameters. NIW requests with higher token counts can interfere with IW request execution, which is handled by chunking requests into fixed-size jobs. Therefore, we assume NIW request token counts are comparable to other workload tiers.

6.3 Forecast Logic and Optimization Module

For IW requests, the *Load Predictor* (Fig. 2) forecasts the input TPS per region and per model type using the popular ARIMA time-series forecasting model [43]. As discussed in § 5, IW workloads are predictable and we find ARIMA to be accurate enough to forecast the diurnal load for each model in a region. We take the maximum TPS expected in the next hour from the forecast to estimate the TPS capacity required by the model to serve the IW load. We add a buffer of β to this forecasted TPS to handle transient bursts and to offer spare capacity for NIW requests. We set the buffer as $\beta = 10\%$ of the NIW load received in the past hour. The output of the forecast is sent to the optimization module, which uses an ILP solver to solve the optimization problem (§ 5). This returns $\delta_{i,j}$, the change in the number of instances assigned to model i at region j , which is passed to the Scaling Logic. The forecast and optimization modules run each hour.

Choice of Forecast Model. We choose an ARIMA model for forecasting the input TPS. We also explore four alternate models, with ARIMA providing accurate predictions with a low training and prediction latency, crucial to the performance of our system. We discuss these in Appendix B.

6.4 Scaling Logic

The long-term scaling logic, *Autoscaler* (Fig. 2), uses the hourly instance change recommendations, $\delta_{i,j}$, for a specific region and model type to *scale out* the instances, if $\delta_{i,j} > 0$; and *scale in* instances if $\delta_{i,j} < 0$. For scale out, we first reclaim spot instances of the identical model type which are faster to acquire, and if none are available, we reclaim spot instances from other model types which can be slower to provision. Similarly, for scaling down, we donate the instances to the spot instances of the same model type. We have a few choices on when to initiate the scaling.

Immediate (LT-I). One naive approach is to instantly scale to the instance count recommended by the scaler every hour, which we call Immediate (LT-I). However, the recommendation is based on the peak load that will occur in the next 1 hour. So scaling out immediately can cause transient over-provisioning, well before the peak load actually arrives. So, we offer two additional deferred strategies to pace the rate at which the deployment state catches up with the forecasted load. These can improve the utilization of VMs and utility of serving requests, making spare capacity available to other models that need it in the same region.

Deferred – Instance Utilization (LT-U). We scale out only when the effective memory utilization actually starts increasing and goes over a threshold of 70% used in our experiments. We keep increasing the instance count as long as this threshold is breached and until we achieve the instances count suggested by the optimization model. Similarly, when downscaling, we do so when the utilization goes below a 30% threshold, again until we have reduced to the suggested number of instances. The region endpoint reports the effective memory usage when it receives a new request.

Deferred – Instance utilization and ARIMA gap (LT-UA). Our optimization model can make erroneous recommendations if the ARIMA predictions are inaccurate, which can happen with bursty requests. As for LT-U, in LT-UA we defer the scale out or in based on the memory usage thresholds actually being breached in either direction. However, we do not strictly stop the scale out or in if the instance count reaches the target count. Instead, during the last 20 mins of the hour, if we have reached the scale out instance count and the observed TPS load is $\geq 5\times$ of what ARIMA predicted, then we continue to scale up the instance count (ARIMA highly underestimated). On

the other hand, if the TPS load received is $\leq 0.5\times$ of what the ARIMA prediction, we continue scaling down (ARIMA highly overestimated). So, we switch from a memory-based strategy to a traffic-based strategy based on observed demand.

6.5 Scheduling Logic

The *Request Scheduler* (Fig. 2) differentiates between fast and normal IW tiers to optimize the SLA. We define d_r as the remaining time until the TTFT deadline for request r , where $d_r < 0$ indicates an expired deadline. The scheduler iterates through requests in the waiting queue, adding as many as possible to the current batch based on available GPU memory. The iteration order determines the request priority. We evaluate four scheduling policies for request ordering:

First Come First Serve (FCFS) orders requests by arrival timestamp, serving earlier arrivals first regardless of SLA tier or deadline urgency. This policy serves as our baseline. *Earliest Deadline First (EDF)* sorts requests by their d_r values in ascending order. Requests with expired deadlines ($d_r < 0$) are prioritized to prevent starvation. This naturally places IW-F requests ahead of IW-N requests arriving simultaneously, given IW-F's stricter TTFT. *Priority First (PF)* processes all IW-F requests in FCFS order before considering any IW-N requests. This policy maximizes IW-F user experience by ensuring absolute priority over IW-N requests. Lastly, *Deadline and Priority Aware (DPA)* provides a configurable policy that balances service quality across both tiers. The algorithm partitions requests into four deadline-based categories: (i) *severely expired* requests with $d_r < -\tau_n$, (ii) *recently expired* requests with $-\tau_n \leq d_r < 0$, (iii) *urgent requests* approaching deadline with $0 \leq d_r \leq \tau_p$ and (iv) *non-urgent requests* with $d_r > \tau_p$. DPA scheduling priority follows this order: (1) severely expired requests to prevent starvation, (2) urgent IW-F requests, (3) urgent IW-N requests, (4) non-urgent IW-F requests, (5) non-urgent IW-N requests, and finally (6) recently expired requests. The threshold parameters τ_n and τ_p enable fine-tuning the balance between tier prioritization and fairness. While this is illustrated for two IW tiers, a similar approach can be extended to additional such latency tiers as well, in future.

7 Evaluation

Our detailed evaluation of SAGESERVE tests its ability to utilize GPU resources while maintaining latency targets for IW and NIW (§ 7.2.1, 7.2.2); Compares it with SOTA scaling strategies on cost advantages (§ 7.2.3, 7.2.4); Evaluates its responsiveness and SLA maintenance for load bursts and prediction errors (§ 7.2.7); and scaling over a week-long period with diurnal patterns (§ 7.2.7).

7.1 Implementation and Simulating Setup

Experimenting with different scaling mechanisms, routing methods, and scheduling policies across multiple LLM types and GPU VMs can be costly in production. To enable scalable validation, we extend the SOTA LLM simulator *Splitwise* [35] to simulate our datacenter setup running multiple models on various hardware. Splitwise models components of a single LLM instance using Python-based discrete event simulation, including request queues. It uses a robust interpolation-based performance model from real inference traces to predict batch inferencing time per model, calculating user metrics like TTFT and E2E latencies. A single Splitwise instance equals one model instance deployment in a region. We verify simulator accuracy against real hardware deployments. Given the proprietary nature of O365's GPT models, we profile open-source models: Bloom-176B, Llama3.1-8B, Llama3.2-3B, and Llama2-70B on H100-80GB VMs with various input/output sizes [35]. The simulator reports TTFT, TBT, E2E per request, and machine-level metrics using trained performance models. Estimates compared on an 80:20 train:test dataset split show MAPE of $<3\%$ (Fig. 9). Using this as an atomic model instance, we build our evaluation harness with multiple instances, a unified event queue, account routing, and model iterations. We further simulate

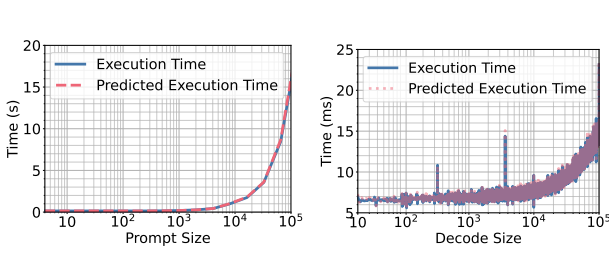


Fig. 9. Comparison of batch execution time predicted by Splitwise vs Real model instance for prompt phase (left) and decode phase (right). The R^2 value is 0.99 and 0.83 for prefill and decode phases respectively. We can infer from the plot that Llama 2 has a prompt TPS of 21000 for decode tokens.

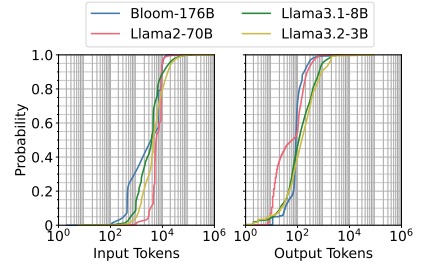


Fig. 10. CDF of Prompt, Output and Total Token Counts in log scale. Production traces for GPT models are mapped to open source LLMs used in evaluation.

multiple global regions to mimic Fig. 2. Our simulator is open-sourced for realistic evaluation of new routing and batching algorithms.

Infrastructure Configuration and Hyper-parameters. We select LLM infrastructure parameters in our simulations, such as utilization thresholds, to match production choices at O365, as discussed in the methods. The minimum instance count in a deployment is 2, and the maximum is 3. For around 90% of regions, network latency is within 500ms with less than 2% of cases having latency of 2.5s. We select ARIMA hyper-parameters using AIC testing. ARIMA adds $\sim 0.7s$ overhead for next-hour predictions and the ILP solver adds $\sim 1.5s$ overhead for optimal instance allocation. SAGESERVE's intelligence is embedded within the controller module of the simulator (Fig. 2), which coordinates with different regions. Network overheads between regions affecting client request latency when routed to instances in different regions are captured using real latency distributions.

GPUs and LLM Models. We evaluate workloads for the three US regions, and four standard LLM models, Bloom, Llama-2, Llama-3.1 and Llama-3.2, used by IW and NIW with their default weights. All the model types are assigned 20 instances per region at the start. We assume homogeneous hardware and set the GPU cards needed by each model as identical. The TPS capacity for each LLM instance on each GPU type is shown in Fig. 9. We assume the redeployment time for an LLM model with weights available in the same region as 10 minutes for all the models regardless of their parameter size, while the redeployment time of a model for which weights are absent in the local region is ≈ 2 hours. These are consistent with our observations from O365. For spot instances being reclaimed, it takes a median of 1 minute and a maximum of 5 minutes. The simulator also handles unavailability of VMs being re-provisioned.

Workload. We use the July, 2025 trace by default in our experiments, but also confirm that the observed results are consistent with the Nov, 2024 traces. Fig. 10 provides a distribution of the input and output token counts for 4 internal models, which we map to the open source models we evaluate. In general, the majority of requests have an input token count $> 1k$, while most output token counts are $< 1k$, but vary with the model type.

Baselines. Unified Reactive Heuristic (Reactive, § 4) baseline represents the current deployment at O365. Whenever a request arrives at the regional endpoint of a model type, the effective utilization is measured, and if the value is greater than 70% ($< 30\%$), the instance is scaled out (in), with a cooldown period of 15s between scaling events. For SOTA baseline, we implement Chiron [36] in our simulator. We initialize the system with ten interactive, five mixed and five batch instances of each model type in every region and set $\Theta = 0.6$ in its interactive autoscaling algorithm for ideal performance on our traces, as guided by their work. We keep the rest of the components like

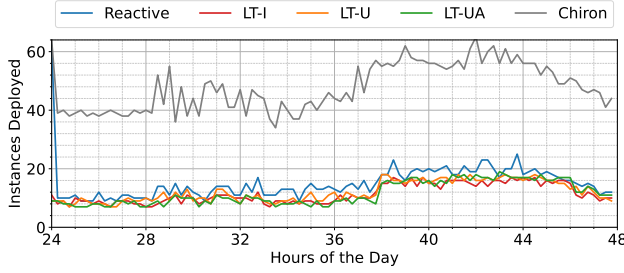
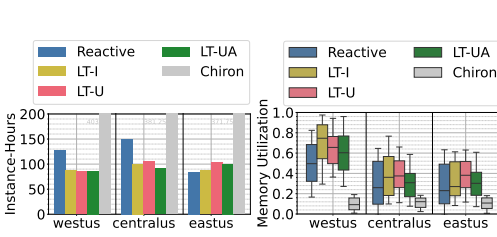


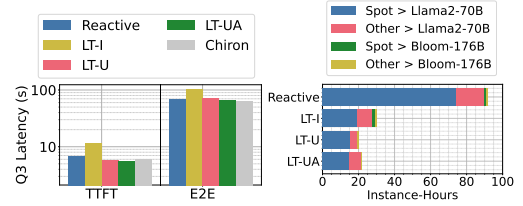
Fig. 11. Aggregated sum of instances deployed across regions for Llama-2 on a peak traffic day. Area under curve for Reactive, LT-I, LT-U, LT-UA and Chiron are 362.25, 274.5, 291, 277.5 and 1146, instance-hours. This translates to a savings of $\approx \$0.6M$ per week.

global routers and schedulers consistent between Chiron and SAGESERVE for a fair comparison. We contrast the baselines with four strategies of SAGESERVE: LT-I, LT-U, and LT-UA.



(a) Model Instance Hours (b) Memory Utilization

Fig. 12. Llama-2 results for different strategies and regions.



(a) 75%tile Latency Metrics (b) GPU hours wasted on scaling

Fig. 13. Llama-2 results on a peak traffic day. (b) shows the time to acquire spot instances.

7.2 Results

7.2.1 Effectiveness of Proactive Strategies in Reducing GPU-hours. We first evaluate the effectiveness of SAGESERVE for a single day trace. Figure 11 shows the trends of instance hour usage by hour, aggregated across all the three regions for Llama-2 70B model. Our forecast aware strategies consistently use less instances as compared to the reactive strategy, with LT-I, LT-U and LT-UA using 24.21%, 19.65% and 23.38% less instance hours respectively. This is intuitive as LT approaches do not react to momentary bursts in traffic and scale based on the forecasts. These results are also evident in Fig. 12a, which shows LT strategies are better for all regions. At the time of writing, the cost of an H100 cluster on Azure is \$98.32/hour. Saving 85 instance hours a day for a single model in a single region would translate to roughly \$0.6M ($\$98.32 \times 85 \times 3 \text{ models} \times 4 \text{ regions} \times 7 \text{ days}$) per week in our setting, or about \$2.5M per month!

7.2.2 GPU Cost Reduction without SLO Violations. LT-I utilizes less GPU hours but it slightly harms the TTFT and E2E latency of requests, as evident from Figure 13a. This is because while immediately scaling up does not significantly benefit requests when traffic is lower, immediate scale down slows down requests, potentially harming their SLOs. These issues are fixed in LT-U and LT-UA, where instances scale on demand. These strategies help us downscale only if we can do so while serving low latency requests, while ensuring we do not up (or down) scale too much.

7.2.3 Comparison with Chiron. Chiron optimizes solely for SLA, but can lead to increased instance demand without clear tail latency improvements (Figures 11 and 13a). Chiron also exhibits lower hardware utilization compared to us and even our reactive scaling mechanisms. This stems from its reliance on offline profiles for infrastructure scaling rather than online memory usage metrics.

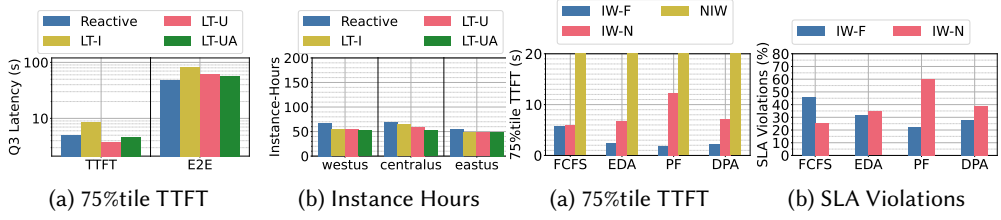


Fig. 14. Results after adding Llama-4 Scout as a fifth model to our experiments. Fig. 15. (a) Latency and (b) SLA violation for workloads as we change the scheduling policy.

7.2.4 Scaling Costs. Due to the large amount of time for LLM instances to initiate even after GPU VMs have been acquired, frequent scale up events can result in increase in GPU-hours without any benefits to request serving. In Figure 13b SAGESERVE is able to reduce the GPU cycles wasted during scale up events by about 70%. Due to fluctuations in traffic, reactive scaling generally scale down unnecessarily, wasting GPU cycles when redeployment is inevitably needed. Using the forecasts, SAGESERVE reduces the number of times we upscale, resulting in better hardware usage.

7.2.5 Scalability Test. We evaluate the generalizability of our SAGESERVE by incorporating a fifth Llama 4 Scout model into our experiment, with 109B parameters but utilizing a Mixture-of-Experts (MoE) architecture. MoE's efficiency gains are reflected across our key performance metrics. Despite Scout's larger parameter count, we see substantial improvement in latency (Fig. 14a) while maintaining high memory utilization. SAGESERVE's benefits persist even after an increase in the baseline memory utilization of the Reactive approach. The higher throughput of Scout also results in a fewer instance hours for the model (Fig. 14b). Hence, SAGESERVE scales effectively for diverse model architectures, adapting to unique efficiency characteristics of MoE models while maintaining the benefits for traditional dense models.

7.2.6 Performance of Multi Tier Workloads. Examining IW-F and IW-N, the default setting fails to distinguish between their different SLAs. Treating both the workloads similarly gives them similar Q3 TTFT (~ 5.6s, Fig. 15a) but results in much higher SLA violations for IW-F (~ 45%) than IW-N (~ 25%, Fig. 15b). We test three different scheduling approaches of SAGESERVE to handle the SLA tiers. EDF scheduling balances the violations more evenly (31% for IW-F and 34% for IW-N) by reducing the Q3 TTFT for IW-F to 2.4s while increasing IW-N Q3 TTFT to 6.1s. PF scheduler achieves the lowest violations for IW-F (24% with .9s response time) at the significant expense of IW-N (60% violations and 12.1s TTFT). DPA provides a middle ground with 28% and 38% violations and 2.1s and 7.9s Q3 TTFT for IW-F and IW-N respectively. It can be further tuned to favor either workload types. All three schedulers can be extended to support additional SLA tiers. Cloud providers could implement SLA differentiation at the routing level as well by dedicating specific instances to high-priority workloads, though we leave this for future work.

7.2.7 Burst Management using SAGESERVE. We further evaluate the responsiveness to spikes by randomly increasing the incoming load to 8x to simulate sudden traffic bursts (blue curve, Fig. 16a). While LT-U and LT-I maintain their latency and memory usage for small bursts, they do not scale above the threshold set by the ILP and the ARIMA forecast even for large bursts. This is evident in the peak latencies during this time, where the green curve of LT-UA is able to reduce back its memory usage faster than LT-I and LT-U. So, in such scenarios, LT-UA copes with the uncertainty much better. As discussed in § 6.4, we set the threshold to scale up at 5x predicted traffic.

Validation on Week Long Trace. Fig. 16b displays the 95tile of TTFT and E2E latency over the course of one week. The insights gained from a one-day trace also apply in this case. The reactive strategy shows inferior performance, while other strategies achieve better performance metrics.

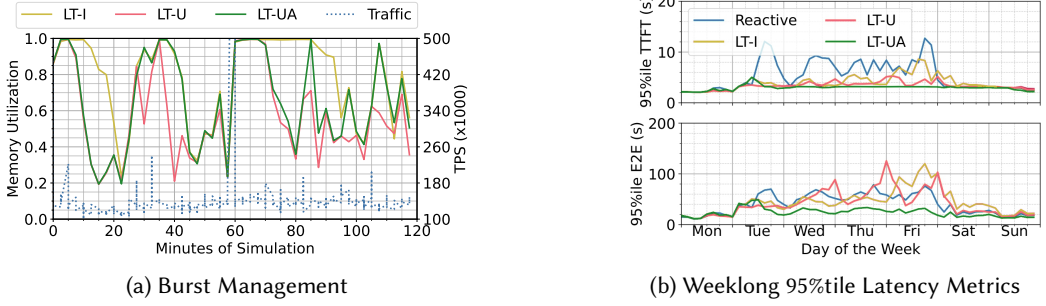


Fig. 16. (a) Performance of LT-UA with synthetic request bursts. (b) P95 latency binned by 3h for Llama2-70B.

LT-U and LT-UA behave similarly during weekdays, with a slight change in performance at the start of the weekend. This indicates the effect of the LT-UA strategy across longer time scales, which accounts for errors in ARIMA forecasts when the trend in TPS differs during the weekend. Overall, SAGESERVE scales well with longer traces and for different request rates.

Validation for Nov, 2024 Trace. We observe similar trends while evaluating using the earlier trace. Instance hours for Llama 2 on a peak traffic day are 302, 227, 248 and 233 for Reactive, LT-I, LT-U and LT-UA, with a 25% reduction in instance hours seen for us. This validates the generalizability of our methods for production traces across time.

Ablation Studies. We test the robustness of our system in scenarios with varying load distribution and with different hardware serving the LLMs as well. Our method generalizes well over these scenarios and we discuss them in [Appendix A](#).

8 Related Work

Autoscaling for cloud computing. There is a long history of forecast-based autoscaling for CPU VMs [17, 39, 40, 47], illustrating its necessity in cloud services, challenges at cloud scale, and potential cost savings. For example, [17] shows that even 1% reduction in VM fragmentation yields \$100M annual savings, with higher expected savings for GPUs due to their greater cost [35]. Our work differs as we consider autoscaling LLMs on GPUs, which faces unique challenges: high latency sensitivity of interactive workloads, significant SLA variation across workloads, and high cost and latency of migrating and loading LLMs (hundreds of GB) on GPUs. Additionally, the unique nature of LLM computation with different prefill (compute bound) and decode (memory bound) phase characteristics must be considered when autoscaling. *Efficient LLM routing and serving.* [37] discusses daily peaks, off-peaks, and unpredictable spikes in LLM inference workloads. Several works optimize latency or throughput at single model instances, including efficient attention computation [12, 22], batched inference [5, 54], QoE optimization [23], and fair service [42]. However, these consider only single LLM instances and do not account for multiple model types and GPUs. [19] assumes identical LLM types and equal-priority workloads, focusing on load balancing across regional instances and addressing performance interference. [13] leverages additional storage and memory for faster LLM loading and live migration. While routing and scaling strategies exist for general workloads and LLMs, they often make simplifying assumptions.

LLM autoscaling. Since the advent of commercial LLM serving platforms [1–3], research has explored autoscaling LLM resources to meet workload requirements. [15, 21, 25] explored optimal model placement on GPUs through various optimization formulations. However, these optimizations are static and do not consider dynamic workloads, rendering them ineffective for the high traffic fluctuations observed in commercial platforms (Fig. 5). Other works address specific serving challenges like pre-emption [27] and startup/migration delays [13], but do not consider the diverse

workload mix and varying model resource requirements that SAGESERVE does. Some works consider heterogeneous workloads but use reactive approaches: Llumnix[45] performs fine-grained context-switching for GPU memory defragmentation and load balancing, while ConServe[38] pre-empt NIW requests to prioritize interactive requests within single instances. Chiron [36] performs backpressure-based autoscaling to optimize latency metrics for interactive requests across clusters with heterogeneous workloads. These works focus on regional-level deployments, overlooking inter-region imbalances and load disparities across LLM types within regions. Using key workload insights, SAGESERVE proposes that scheduling and routing should consider the system holistically to better utilize available capacity. Finally, [32, 53] consider routing requests across model instances for latency minimization or throughput maximization. While request routing handles short-term traffic fluctuations, it must be combined with dynamic model scaling as in SAGESERVE to handle longer-term variations and avoid stale assignments.

LLM workloads and simulators. While prior works such as Splitwise [35], Vidur [4], and Burst-GPT [50] have contributed datasets and simulators for LLM workloads, our simulator models the entire inference stack—from regions to deployments to models—within a heterogeneous environment. Unlike [35], which simulates only single model instances, and [4], which lacks heterogeneous deployment and regional abstractions, our framework enables cloud-scale simulation across diverse hardware and model types. Datasets from prior works focus on narrow use cases like summarization [10] or chat [35, 49, 50], whereas our trace—sourced from Microsoft Copilot—captures enterprise LLM interactions across Microsoft365 apps like Word, Excel, PowerPoint, Teams, and Outlook, incorporating RAG-based inputs that diversify input and output distributions. Additionally, datasets such as [4, 35, 50] are collected at regional levels and include only basic attributes like request time, input/output length, and LLM type. These limitations hinder capturing scaling challenges in realistic, global production environments. We extensively characterize workload tiers with different SLAs, their request-level characteristics, and the distribution of workloads and LLM types across multiple regions, using these insights to guide cost-effective solutions at scale. Lastly, inference engines like vLLM [20] and Sarathi-Serve [5] can run LLMs on GPUs to serve workloads but are not designed for simulation-based capacity planning, which is central to our work.

9 Conclusions

In this paper, we characterize LLM inferencing workloads from production traces of Microsoft O365, revealing insights on temporal behavior across latency tiers and regions. We use this to design SAGESERVE, a holistic system for serving LLM inference requests with diverse SLAs that maintains better GPU utilization, reduces resource fragmentation from silos, and increases utility by donating surplus instances to Spot instances. SAGESERVE achieves this through its holistic deployment stack for varying SLA requests, async feed module for NIW, and long-term aware proactive scaler logic that capitalizes on underutilized instances through inter-model redeployment. These benefits are confirmed through experiments with a realistic simulator and our traces against baseline and SOTA methods, potentially saving millions of USD monthly.

As lessons learned, theoretical performance limits for LLM types differ from real-world achievements. Cost-effective solutions at scale require extensive performance benchmarking and deep production insights. Future work includes extending SAGESERVE to accommodate workloads with a continuum of SLAs and conducting studies on the proposed approach with deployments across heterogeneous hardware types.

10 Acknowledgements

We would like to thank all the reviewers for their constructive feedback and our shepherd, Prof. Jian Li for helping us incorporate all the reviews and improving the final version of our work.

References

- [1] [n. d.]. ChatGPT. <http://chat.openai.com>.
- [2] [n. d.]. Copilot. <http://copilot.microsoft.com>.
- [3] [n. d.]. Gemini. <http://gemini.google.com>.
- [4] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S Gulavani, Ramachandran Ramjee, and Alexey Tumanov. 2024. Vidur: A large-scale simulation framework for llm inference. *Proceedings of Machine Learning and Systems* 6 (2024), 351–366.
- [5] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming {Throughput-Latency} Tradeoff in {LLM} Inference with {Sarathi-Serve}. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 117–134.
- [6] "AWS". "2024". "Introducing Fast Model Loader in SageMaker Inference: Accelerate autoscaling for your Large Language Models (LLMs)". <https://aws.amazon.com/blogs/machine-learning/introducing-fast-model-loader-in-sagemaker-inference-accelerate-autoscaling-for-your-large-language-models-llms-part-1/>.
- [7] BigScience. [n. d.]. Introducing The World's Largest Open Multilingual Language Model: BLOOM [Online]. In <https://bigscience.huggingface.co/blog/bloom>.
- [8] Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, Erik Brynjolfsson, S. Buch, Dallas Card, Rodrigo Castellon, Niladri S. Chatterji, Annie S. Chen, Kathleen A. Creel, Jared Davis, Dora Demszky, Chris Donahue, Moussa Doumbouya, Esin Durmus, Stefano Ermon, John Etchemendy, Kavin Ethayarajh, Li Fei-Fei, Chelsea Finn, Trevor Gale, Lauren E. Gillespie, Karan Goel, Noah D. Goodman, Shelby Grossman, Neel Guha, Tatsunori Hashimoto, Peter Henderson, John Hewitt, Daniel E. Ho, Jenny Hong, Kyle Hsu, Jing Huang, Thomas F. Icard, Saahil Jain, Dan Jurafsky, Pratyusha Kalluri, Siddharth Karamcheti, Geoff Keeling, Fereshte Khani, O. Khattab, Pang Wei Koh, Mark S. Krass, Ranjay Krishna, Rohith Kudithipudi, Ananya Kumar, Faisal Ladhak, Mina Lee, Tony Lee, Jure Leskovec, Isabelle Levent, Xiang Lisa Li, Xuechen Li, Tengyu Ma, Ali Malik, Christopher D. Manning, Suir P. Mirchandani, Eric Mitchell, Zanele Muniyikwa, Suraj Nair, Avaniika Narayan, Deepak Narayanan, Benjamin Newman, Allen Nie, Juan Carlos Niebles, Hamed Nilforoshan, J. F. Nyarko, Giray Ogut, Laurel Orr, Isabel Papadimitriou, Joon Sung Park, Chris Piech, Eva Portelance, Christopher Potts, Aditi Raghunathan, Robert Reich, Hongyu Ren, Frieda Rong, Yusuf H. Roohani, Camilo Ruiz, Jack Ryan, Christopher R'e, Dorsa Sadigh, Shiori Sagawa, Keshav Santhanam, Andy Shih, Krishna Parasuram Srinivasan, Alex Tamkin, Rohan Taori, Armin W. Thomas, Florian Tramèr, Rose E. Wang, William Wang, Bohan Wu, Jiajun Wu, Yuhuai Wu, Sang Michael Xie, Michihiro Yasunaga, Jiaxuan You, Matei A. Zaharia, Michael Zhang, Tianyi Zhang, Xikun Zhang, Yuhui Zhang, Lucia Zheng, Kaitlyn Zhou, and Percy Liang. 2021. On the Opportunities and Risks of Foundation Models. *ArXiv* (2021). <https://crfm.stanford.edu/assets/report.pdf>
- [9] "Google Cloud". "2025". "From LLMs to image generation: Accelerate inference workloads with AI Hypercomputer". <https://cloud.google.com/blog/products/compute/ai-hypercomputer-inference-updates-for-google-cloud-tpu-and-gpu>.
- [10] Arman Cohan, Franck Dérioncourt, Doo Soon Kim, Trung Bui, Seokhwan Kim, Walter Chang, and Nazli Goharian. 2018. A Discourse-Aware Attention Model for Abstractive Summarization of Long Documents. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*. 615–621.
- [11] Hao Cui, Zahra Shamsi, Gowoon Cheon, Xuejian Ma, Shutong Li, Maria Tikhonovskaya, Peter Norgaard, Nayantara Mudur, Martyna Plomecka, Paul Raccuglia, Yasaman Bahri, Victor V. Albert, Pranesh Srinivasan, Haining Pan, Philippe Faist, Brian Rohr, Michael J. Statt, Dan Morris, Drew Purves, Elise Kleeman, Ruth Alcantara, Matthew Abraham, Muqthar Mohammad, Ean Phing VanLee, Chenfei Jiang, Elizabeth Dorfman, Eun-Ah Kim, Michael P Brenner, Viren Jain, Sameera Ponda, and Subhashini Venugopalan. 2025. CURIE: Evaluating LLMs On Multitask Scientific Long Context Understanding and Reasoning. *arXiv:2503.13517 [cs.CL]* <https://arxiv.org/abs/2503.13517>
- [12] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems* 35 (2022), 16344–16359.
- [13] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. 2024. ServerlessLLM: Low-Latency Serverless Inference for Large Language Models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 135–153.
- [14] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. {Cost-Efficient} large language model serving for multi-turn conversations with {CachedAttention}. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. 111–126.
- [15] Tyler Griggs, Xiaoxuan Liu, Jiaxiang Yu, Doyoung Kim, Wei-Lin Chiang, Alvin Cheung, and Ion Stoica. 2024. M\`elange: Cost Efficient Large Language Model Serving by Exploiting GPU Heterogeneity. *arXiv preprint arXiv:2404.14527* (2024).

- [16] Varun Gupta, Mor Harchol Balter, Karl Sigman, and Ward Whitt. 2007. Analysis of join-the-shortest-queue routing for web server farms. *Performance Evaluation* 64, 9-12 (2007), 1062–1081.
- [17] Ori Hadary, Luke Marshall, Ishai Menache, Abhisek Pan, Esaias E Greeff, David Dion, Star Dorminey, Shailesh Joshi, Yang Chen, Mark Russinovich, et al. 2020. Protean: {VM} allocation service at scale. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 845–861.
- [18] Junda He, Christoph Treude, and David Lo. 2025. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–30.
- [19] Kunal Jain, Anjaly Parayil, Ankur Mallick, Esha Choukse, Xiaoting Qin, Jue Zhang, Íñigo Goiri, Rujia Wang, Chetan Bansal, Victor Rühle, et al. 2025. Performance Aware LLM Load Balancer for Mixed Workloads. In *Proceedings of the 5th Workshop on Machine Learning and Systems*. 19–30.
- [20] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.
- [21] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E Gonzalez, et al. 2023. {AlpaServe}: Statistical multiplexing with model parallelism for deep learning serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. 663–679.
- [22] Hao Liu, Matei Zaharia, and Pieter Abbeel. [n. d.]. RingAttention with Blockwise Transformers for Near-Infinite Context. In *The Twelfth International Conference on Learning Representations*.
- [23] Jiachen Liu, Zhiyu Wu, Jae-Won Chung, Fan Lai, Myungjin Lee, and Mosharaf Chowdhury. 2024. Andes: Defining and Enhancing Quality-of-Experience in LLM-Based Text Streaming Services. *arXiv preprint arXiv:2404.16283* (2024).
- [24] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. 2024. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. *arXiv:2408.06292 [cs.AI]* <https://arxiv.org/abs/2408.06292>
- [25] Yixuan Mei, Yonghao Zhuang, Xupeng Miao, Juncheng Yang, Zhihao Jia, and Rashmi Vinayak. 2024. Helix: Distributed Serving of Large Language Models via Max-Flow on Heterogeneous GPUs. *arXiv preprint arXiv:2406.01566* (2024).
- [26] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Hongyi Jin, Tianqi Chen, and Zhihao Jia. 2023. Towards efficient generative large language model serving: A survey from algorithms to systems. *Comput. Surveys* (2023).
- [27] Xupeng Miao, Chunan Shi, Jiangfei Duan, Xiaoli Xi, Dahua Lin, Bin Cui, and Zhihao Jia. 2024. Spotserve: Serving generative large language models on preemptible instances. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 1112–1127.
- [28] Microsoft. 2024. Online endpoint deployment for real-time inferencing. <https://learn.microsoft.com/en-us/azure/machine-learning/concept-endpoints-online>.
- [29] Microsoft. 2024. Run Azure OpenAI models in batch endpoints to compute embeddings. <https://learn.microsoft.com/en-us/azure/machine-learning/how-to-use-batch-model-openai-embeddings>.
- [30] Microsoft. 2025. Azure OpenAI Batch API.
- [31] San Murugesan. 2025. The rise of agentic AI: implications, concerns, and the path forward. *IEEE Intelligent Systems* 40, 2 (2025), 8–14.
- [32] Chengyi Nie, Rodrigo Fonseca, and Zhenhua Liu. 2024. Aladdin: Joint Placement and Scaling for SLO-Aware LLM Serving. *arXiv preprint arXiv:2405.06856* (2024).
- [33] OpenAI. 2024. Batch API. <https://platform.openai.com/docs/api-reference/streaming>.
- [34] OpenAI. 2024. Streaming API. <https://platform.openai.com/docs/api-reference/streaming>.
- [35] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 118–132.
- [36] Archit Patke, Dharmath Reddy, Saurabh Jha, Chandra Narayanaswami, Zbigniew Kalbarczyk, and Ravishankar Iyer. 2025. Hierarchical Autoscaling for Large Language Model Serving with Chiron. *arXiv:2501.08090 [cs.DC]* <https://arxiv.org/abs/2501.08090>
- [37] Ye Qi. 2025. Scaling Large Language Model Serving Infrastructure at Meta. <https://www.infoq.com/presentations/llm-meta/>.
- [38] Yifan Qiao, Shu Anzai, Shan Yu, Haoran Ma, Yang Wang, Miryung Kim, and Harry Xu. 2024. ConServe: Harvesting GPUs for Low-Latency and High-Throughput Large Language Model Serving. *arXiv preprint arXiv:2410.01228* (2024).
- [39] Nilabja Roy, Abhishek Dubey, and Aniruddha Gokhale. 2011. Efficient autoscaling in the cloud using predictive models for workload forecasting. In *2011 IEEE 4th International Conference on Cloud Computing*. IEEE, 500–507.
- [40] Krzysztof Rzadca, Paweł Findeisen, Jacek Swiderski, Przemysław Zych, Przemysław Broniek, Jarek Kusmierek, Paweł Nowak, Beata Strack, Piotr Witusowski, Steven Hand, et al. 2020. Autopilot: workload autoscaling at google. In *Proceedings of the Fifteenth European Conference on Computer Systems*. 1–16.
- [41] P. Schmid, O. Sanseviero, P. Cuenca, and L. Tunstall. [n. d.]. Llama 2 is here - Get it on Hugging Face [Online]. In Available: <https://huggingface.co/blog/llama2>.

- [42] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E Gonzalez, and Ion Stoica. 2024. Fairness in serving large language models. In 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24). 965–988.
- [43] Robert H Shumway, David S Stoffer, Robert H Shumway, and David S Stoffer. 2017. ARIMA models. Time series analysis and its applications: with R examples (2017), 75–163.
- [44] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Esha Choukse, Haoran Qiu, Rodrigo Fonseca, Josep Torrellas, and Ricardo Bianchini. 2025. Tapas: Thermal-and power-aware scheduling for LLM inference in cloud platforms. In Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2. 1266–1281.
- [45] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: dynamic scheduling for large language model serving. In Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation (Santa Clara, CA, USA) (OSDI'24). USENIX Association, USA, Article 10, 19 pages.
- [46] "Top500". "2025". "Top 500 Supercomputing List". <https://www.top500.org/system/180236/>.
- [47] Prateeksha Varshney and Yogesh Simmhan. 2018. AutoBoT: Resilient and cost-effective scheduling of a bag of tasks on spot VMs. IEEE Transactions on Parallel and Distributed Systems 30, 7 (2018), 1512–1527.
- [48] Ao Wang, Shuai Chang, Huangshi Tian, Hongqi Wang, Haoran Yang, Huiba Li, Rui Du, and Yue Cheng. 2021. {FaaSNet}: Scalable and fast provisioning of custom serverless container runtimes at alibaba cloud function compute. In 2021 USENIX Annual Technical Conference (USENIX ATC 21). 443–457.
- [49] Guan Wang, Sijie Cheng, Xianyuan Zhan, Xiangang Li, Sen Song, and Yang Liu. [n.d.]. OpenChat: Advancing Open-source Language Models with Mixed-Quality Data. In The Twelfth International Conference on Learning Representations.
- [50] Yuxin Wang, Yuhan Chen, Zeyu Li, Xueze Kang, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, et al. 2024. Burstgpt: A real-world workload dataset to optimize llm serving systems. arXiv preprint arXiv:2401.17644 (2024).
- [51] "HPC Wire". 2024. "AWS Delivers the AI Heat: Project Rainier and GenAI Innovations Lead the Way". <https://www.hpcwire.com/2024/12/05/aws-delivers-the-ai-heat-project-rainier-and-genai-innovations-lead-the-way/>.
- [52] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. 2024. Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism. In Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles. 640–654.
- [53] Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. 2023. Fast distributed inference serving for large language models. arXiv preprint arXiv:2305.05920 (2023).
- [54] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22). USENIX Association, Carlsbad, CA, 521–538. <https://www.usenix.org/conference/osdi22/presentation/yu>
- [55] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xiong-Hui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. 2025. AFlow: Automating Agentic Workflow Generation. In The Thirteenth International Conference on Learning Representations. <https://openreview.net/forum?id=z5uVAKwmjf>
- [56] Yanjie Zhao, Xinyi Hou, Shenao Wang, and Haoyu Wang. 2025. Llm app store analysis: A vision and roadmap. ACM Transactions on Software Engineering and Methodology 34, 5 (2025), 1–25.

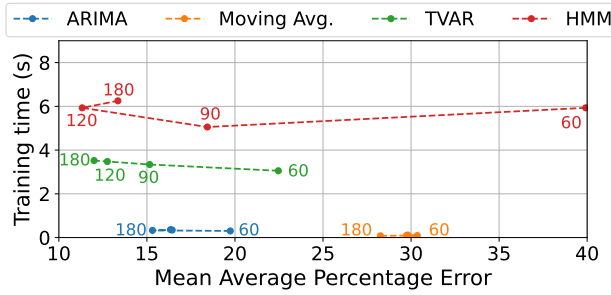


Fig. 17. **Forecast Prediction Models:** Performance of four forecasting models in terms of their mean absolute percentage error and training time. We test these models by training them on four different training windows and study the impact on their training time and accuracy. The moving average method shows poor performance, while the Hidden Markov Method is quite expensive to train, making it an unsuitable choice despite its performance. Both, TVAR and ARIMA are suitable candidates for our design, being fast while being accurate enough. We choose ARIMA with a training window of 60 for our experiments due to its lower training latency.

A Ablation Studies

We conducted experiments with different hardware and different workload distributions to test the robustness of our methods. SAGESERVE (LT-I, LT-U, LT-UA) only requires the token processing profile to be extended to new model hardware pairs. For experiments on A100 clusters, our LT-UA approach uses 28.2% fewer GPU-hours while maintaining tail latency metrics compared to Reactive scaling. This is mainly due to the longer model loading times on A100 clusters. Next, we change the IW to NIW ratio in our traces. In the production traces from Nov. 2024 we see a 3:1 ratio of IW and NIW requests, but our methods are independent of this distribution. After changing this distribution to 9:1 and 1:1, we find that LT-UA requires 26.3% and 22% fewer GPU hours compared to Reactive scaling. These variations are expected, as LT-UA uses the NIW token count to determine the buffer size, which is higher than the original setting in the latter case and lower in the former.

B Forecast Prediction Models

For incoming Tokens Per Minute (TPM) forecasting, we compare ARIMA with the following three additional models along with different window lengths. The training data horizon is 60 minutes.

- (1) Moving Average: This is a simple moving average which predicts the next TPM to be the average of the previous window.
- (2) Time Varying Autoregressive Model (TVAR): This is a popular autoregressive model, which trains on a user-defined window length and makes single value predictions at each time step.
- (3) Hidden Markov Models (HMM): The date, time, and tokens received in the previous window are used as states of the Markov Model to predict the expected incoming TPM over a 60 step horizon.

We compare these methods in their Mean Average Percentage Error in their prediction and latency to train the models. Fig. 17 shows that TVAR and HMM can achieve better prediction accuracy but take much longer training time than ARIMA, while the moving average is quite fast but inaccurate. TVAR with a training window of 90, 120, or 180 would have been a good choice for our system as well, however, we chose ARIMA due to relatively faster training time.

Received July 2025; revised September 2025; accepted October 2025